



Advanced Computational Techniques
for Big Imaging | AA 2025/2026

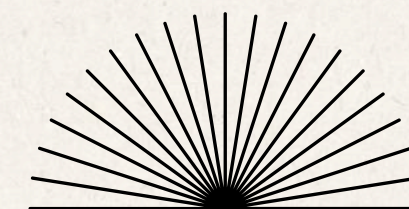
Università degli Studi di Milano-Bicocca

AI SUPER RESOLUTION

Image Upscaler and Enhancer

NAME OF PROJECT:
AI Super Resolution

PRESENTED BY:
Lorenzo Mazzone

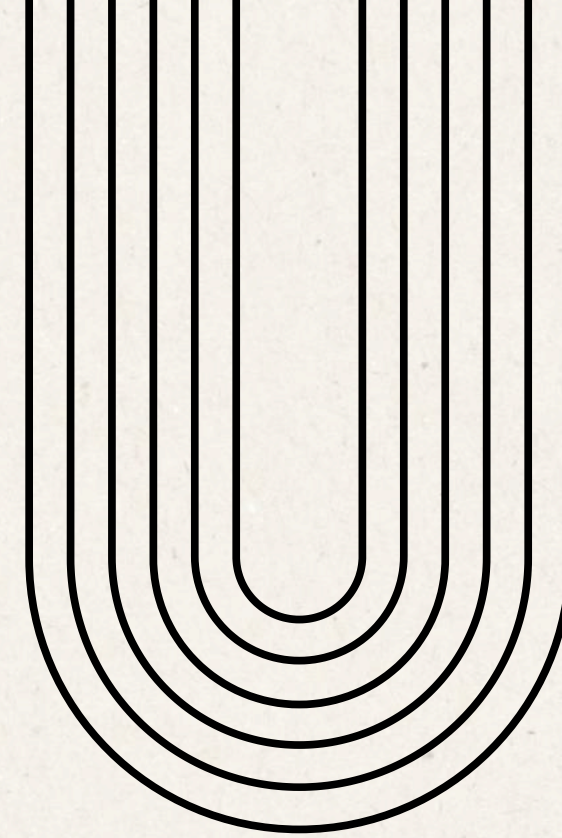


Summary

03	Introduction
04	DIV2K Dataset
05	Preprocessing and Data Augmentation
07	Proposed Solution
09	Architecture, Loss and Metrics
10	Error Analysis and Domain Gap
11	Ablation Study and Domain Alignment
12	Preprocessing and Data Agumentation
13	Results and Comparinson
15	Interactive Demo and UX

Introduction

In the domain of Image Super-Resolution (SR) and Image Restoration, the goal of this project was to reconstruct a high-resolution (HR) image starting from a heavily degraded low-resolution (LR) image (noise, compression artifacts).



Denoise

The first objective is to remove the noise in the image



Enhance

After that, we want to improve the quality of the image, enhancing it

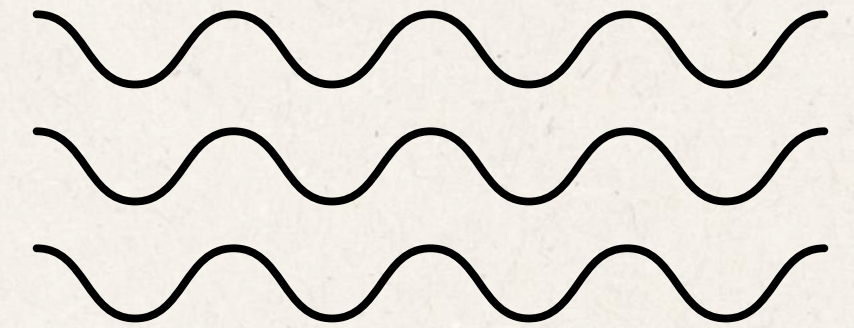


Upscale

Finally, we give as output an upscaled image with a higher resolution



- 01** Cardinality:
800 images for training, 100 for validation
- 02** High-resolution RGB images, containing diverse scenarios rich in high-frequency textures.
- 03** RAM Management:
difficult to load the entire dataset into memory.
- 04** Implemented a Custom PyTorch DataLoader for dynamic loading (lazy loading) and Tiling.

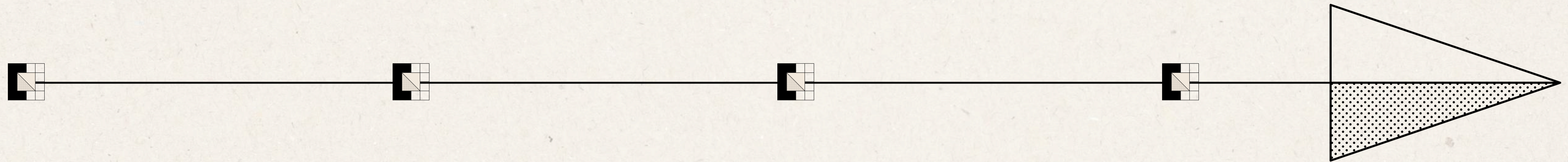


The DIV2K dataset

DIVERse 2K resolution high quality images

Preprocessing and Data Augmentation

How do we modify the dataset?



1

Preprocessing:
Tensor normalization
in the [0, 1] range:

Dynamic extraction
of 256x256 patches to
allow batch training
without running out
of VRAM

2

Geometric Data
Augmentation:

HorizontalFlip and
VerticalFlip to
increase spatial
variance

3

Degradation
Pipeline:

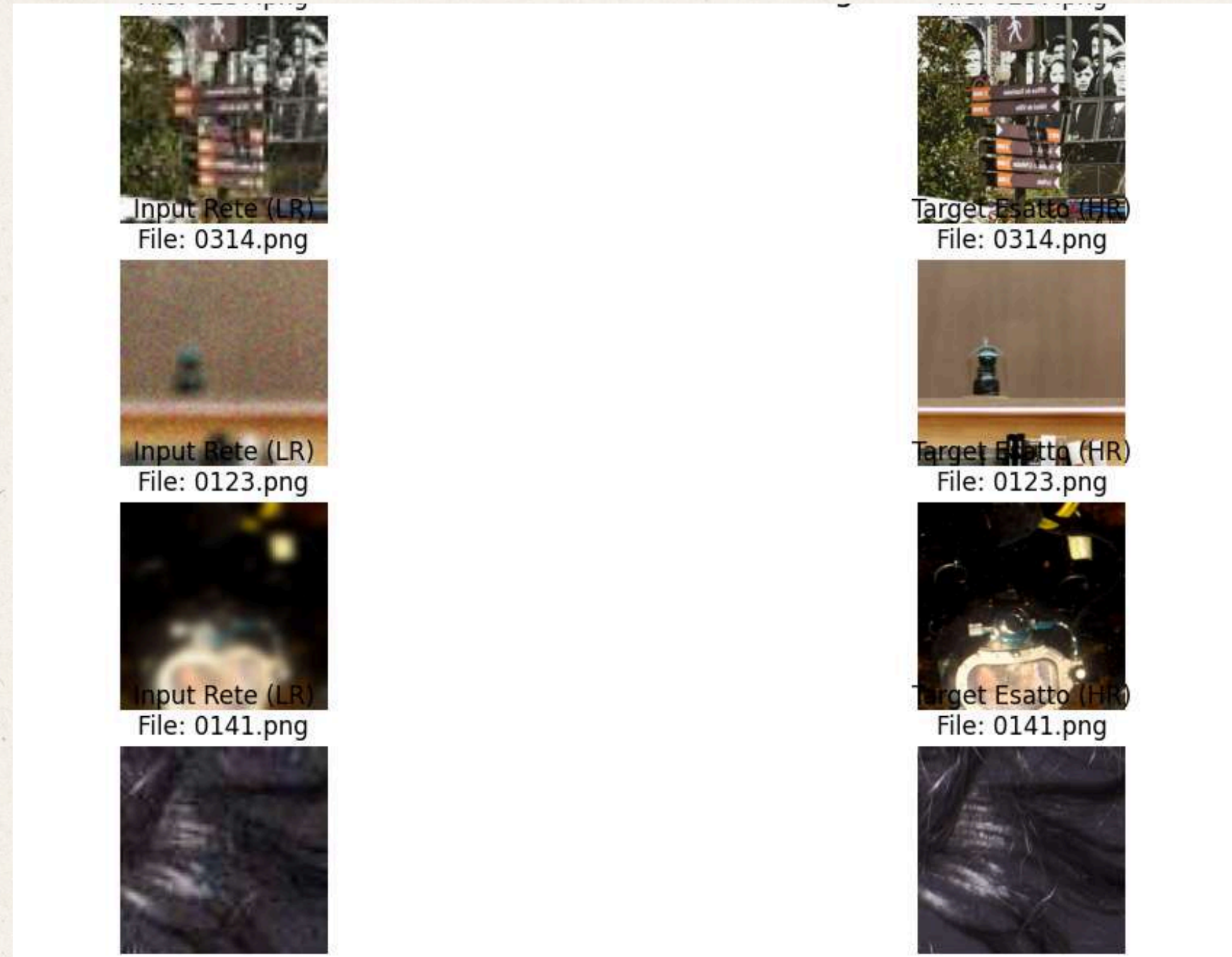
Use of the
albumentations
library to simulate
real-world decay

4

Used techniques:

Downscaling (using
INTER_AREA interpolation to
avoid aliasing),
Addition of GaussNoise,
ImageCompression

Preprocessing and Data Augmentation



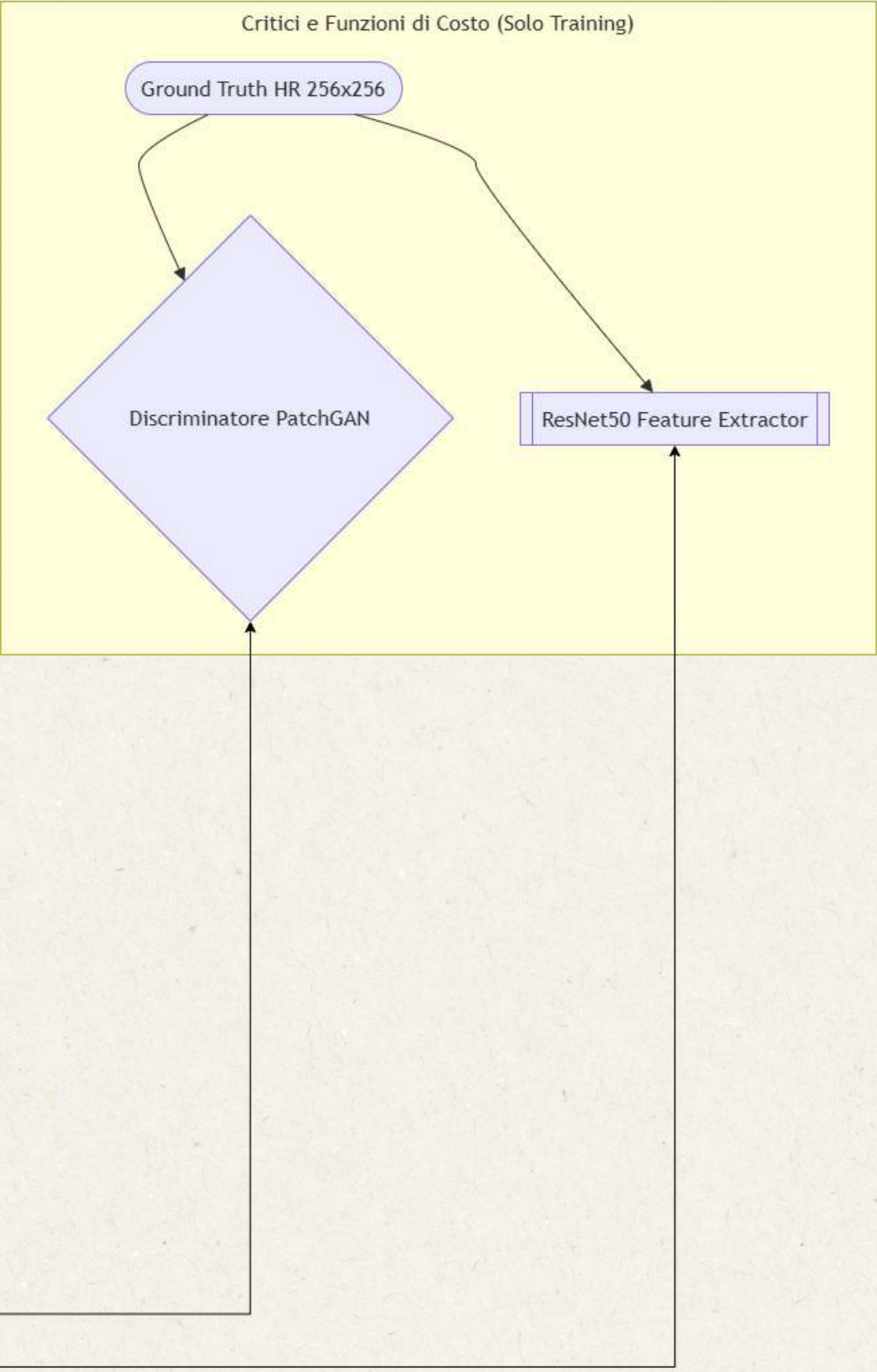
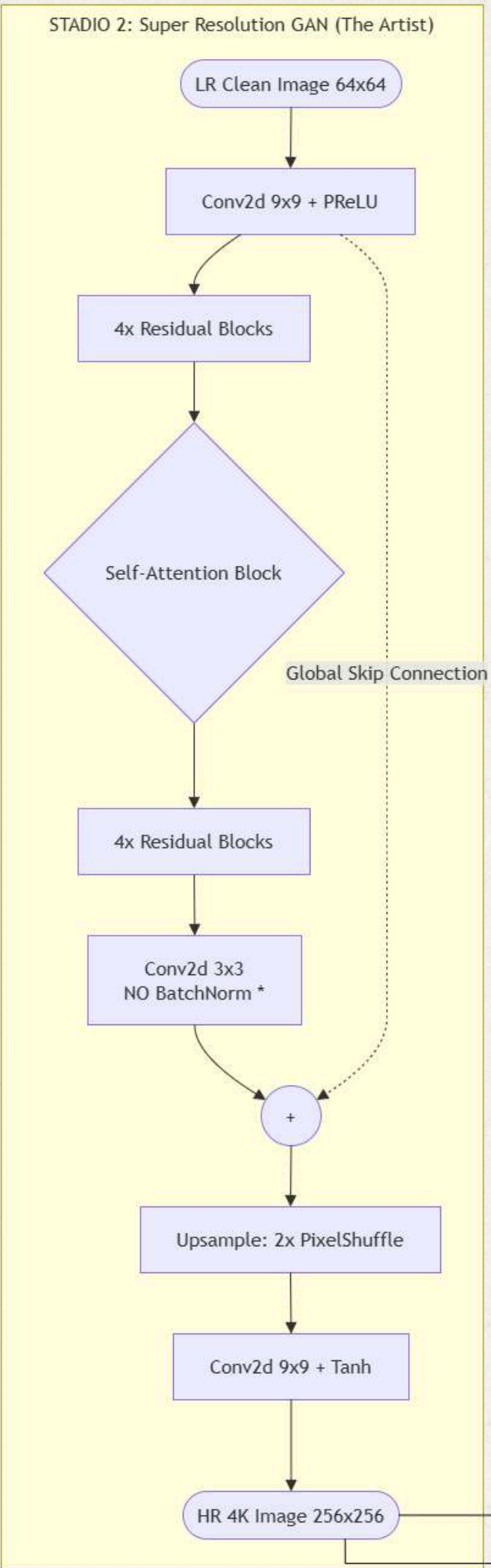
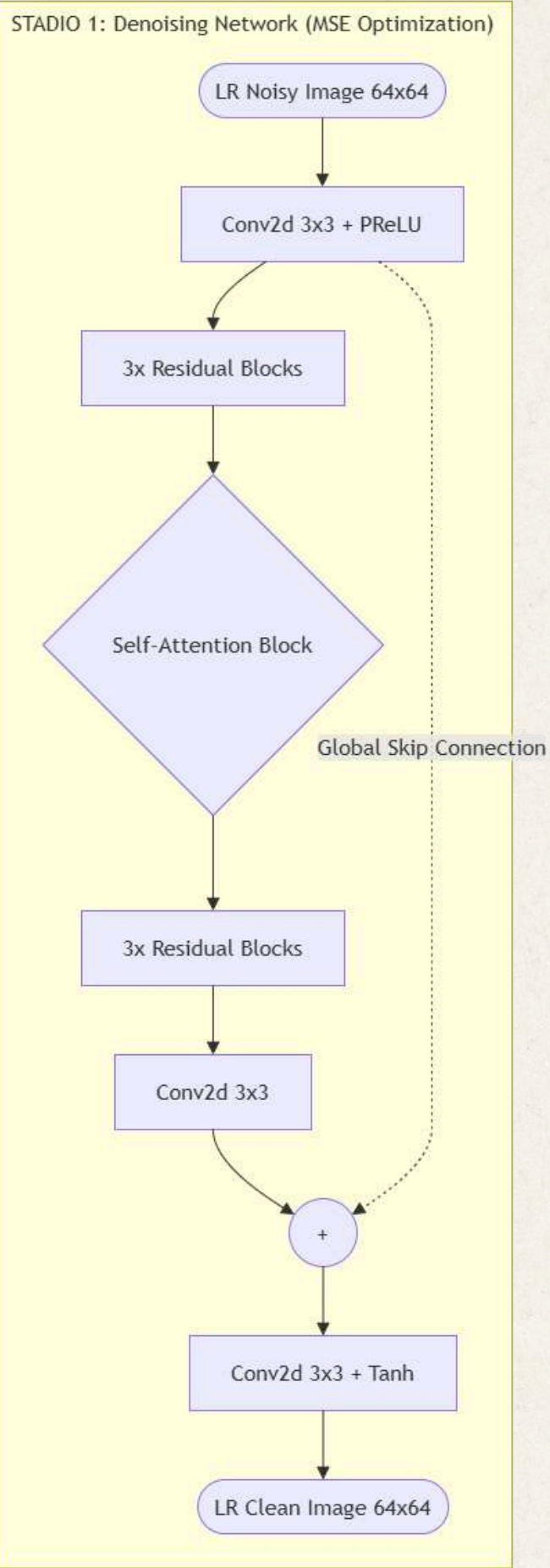
01 Attempt 1: Baseline SRGAN. Mixed results. The network struggled to decide whether to remove noise or enhance details.

02 Attempt 2: Self-Attention Integration (ViT inspired). I integrated a Self-Attention module into the generator to provide global context (inspired by Vision Transformers), but the loss conflict persisted.

03 Final Solution: Decoupled Restoration Pipeline: I separated the problem into two cascaded models: The Denoiser: A network trained strictly to clean the LR image. The Upscaler (SRGAN + Attention): A network trained strictly for upscaling and texture injection on clean images.



**Proposed
solution**



**Proposed
solution**

Architecture, Losses and Metrics

- Networks: Generator (based on Residual Blocks, Attention, and PixelShuffle) + Discriminator (PatchGAN Classifier)
 - Transfer Learning (The Critic): Use of a pre-trained ResNet50 on ImageNet (frozen weights) to extract deep feature maps.
 - The Hybrid Cost Function (Loss):
 - LossPixel (MSE) = Keeps colors stable.
 - LossPerceptual (ResNet) = Forces the network to mimic complex edges and "style".
 - LossAdversarial (BCE) = Fools the discriminator to inject realism.
 - Evaluation Metrics: PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural Similarity).
-

Error Analysis and Domain Gap

- **Problem 1:** Oversmoothing. The dominant use of MSE produced a "watercolor effect".
- **Problem 2:** Checkerboard Artifacts & Collapse. In stage 2, the GAN produced exploded gradients (colorful checkerboards).
- **The Cause (Domain Gap):** The GAN was trained on mathematically perfect synthetic blur. In inference, it received the neural smoothing of Stage 1. The mathematical mismatch caused the network to hallucinate artifacts.



01 Loss Ablation: Decreased the weight of MSE in favor of Adversarial Loss to unlock texture generation.

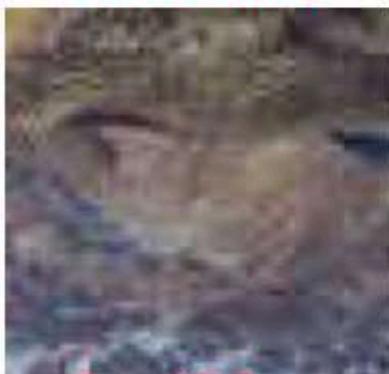
02 Domain Gap Resolution (Teacher Forcing):

- Developed an offline script with Tiling to process the 800 high-resolution images through Network 1.
- The output of Network 1 became the new training input for Network 2.
- Coordinated geometric cropping at the pixel level.

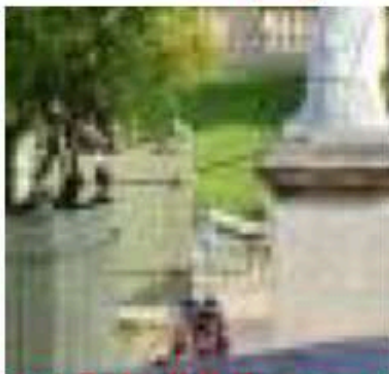


Ablation Study and Domain Alignment

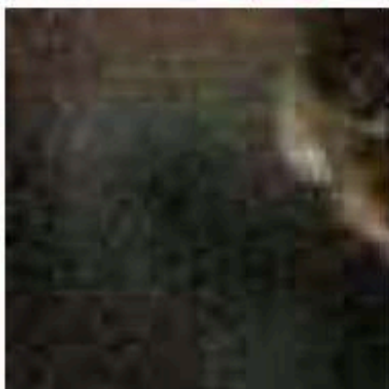
File: 0292.png



Input Rete (LR Rumorosa)
File: 0292.png



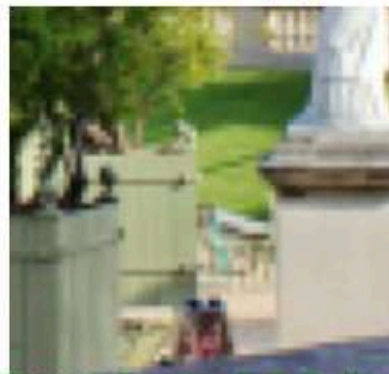
Input Rete (LR Rumorosa)
File: 0722.png



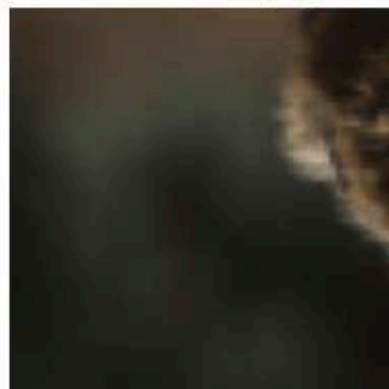
File: 0292.png



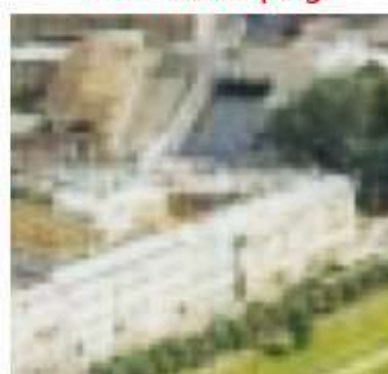
Target Esatto (LR Pulita)
File: 0292.png



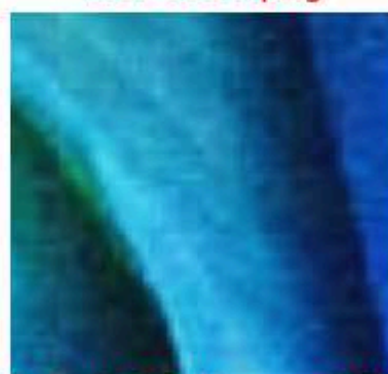
Target Esatto (LR Pulita)
File: 0722.png



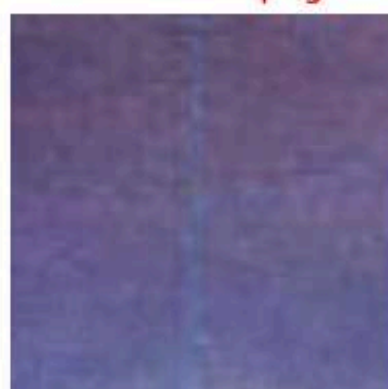
File: 0610.png



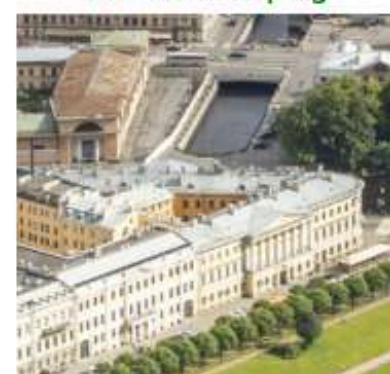
Input Rete (LR Pulita ma PATINATA)
File: 0610.png



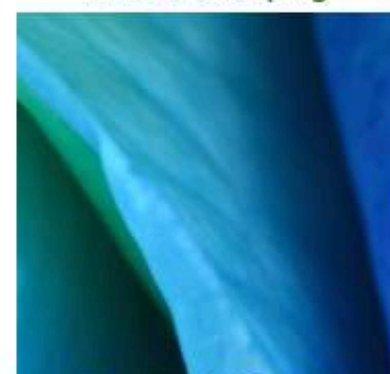
Input Rete (LR Pulita ma PATINATA)
File: 0738.png



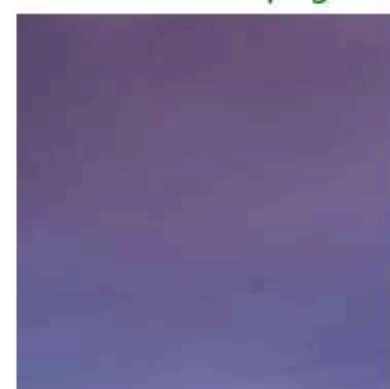
File: 0610.png



Target Esatto (HR Nitida 4K)
File: 0610.png



Target Esatto (HR Nitida 4K)
File: 0738.png





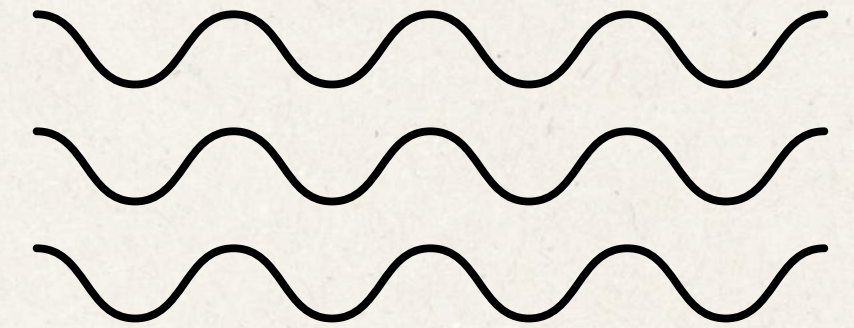
- 01** Simple Baseline:
Bicubic Upscaling
(Results in blurry and
noisy images).



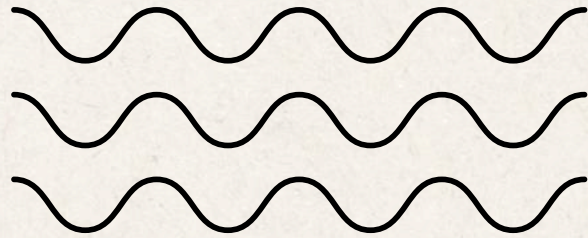
- 02** My Model: Custom
Decoupled Pipeline



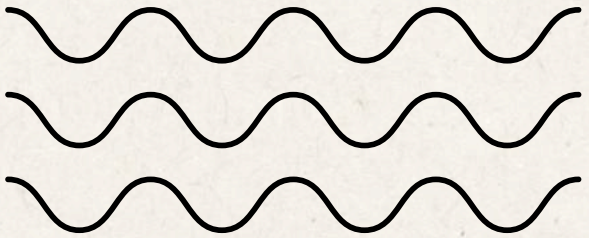
- 03** SOTA: SwinIR X4
(Pre-trained Vision
Transformer).



Results and Comparison (Baselines & SOTA)



Results and Comparison



Model Type	Loss Value	PNSR Value	Similarity Value
Baseline ESPCN	0.0044	23.73	0.74
VGG Model	0.0065	22.32	0.57
VGG Model + ResNet	0.0109	20.39	0.53
VGG Model + Attention	0.0095	20.73	0.54
Decoupled VGG Model - Stage 1	0.0019	27.54	0.83
Decoupled VGG Model - Stage 2	0.0059	22.71	0.59
VGG Model + Attention with Official Dataset	0.004	25.01	0.70

Interactive Demo and UX

- **Framework:** Streamlit UI.
- **Visual Explainability:** Visual Explainability: Allows the user to select every single historical model (Baseline, SRGAN VGG, SRGAN ResNet) to visually evaluate the project's evolution.
- **VRAM Management and Tiling:** VRAM Management & Tiling: Implemented a unified `process_with_tiling` function and explicit garbage collection (`torch.cuda.empty_cache()`) to allow inference on large images/videos without crashing consumer GPUs.
- **SOTA Integration:** Allows testing industrial pre-trained SOTA models (SwinIR Transformer) directly from the interface.

